Experiment - 08

Student Name: Mimanshu Gahlaut
UID: 24BAI70038
Branch: CSE - AIML
Section/Group: 24AIT_KRG-1_B
Semester: 5
Date of Performance: 7/9/26
Subject Name: ADBMS
Subject Code: 24CSP-310

(A)

A company maintains customer information, product information, and sales order information in three separate tables: customer_master, product_catalog, and sales_orders.

The company wants to provide an order report to an external client. The report must contain the Order ID, Order Date, Customer Name, Product Name, and Final Amount.
For every order, first calculate the Gross Amount using:
Gross Amount = Unit Price × Quantity
Then calculate the Discount Amount using:
Discount Amount = Gross Amount × Discount Percentage / 100

Finally, calculate the Final Amount using:

Final Amount = Gross Amount − Discount Amount

Create a complex view using the three tables to generate this report. The view should expose only the required five columns and should not expose the customer's phone/email, product brand/unit price, or the order's quantity/discount percentage.
The company also wants to provide access to an external client. Create a PostgreSQL user named client_user and give this user only SELECT permission on the view. The client must not have direct access to the three original tables and must not be allowed to modify the view data.

Output:

order_id	order_date	full_name	product_name	final_amount
1	2025-09-01	Amit Sharma	Smartphone X100	47500.00
2	2025-09-02	Priya Verma	Laptop Pro 15	58500.00
3	2025-09-03	Ravi Kumar	Wireless Earbuds	15000.00
4	2025-09-04	Neha Singh	Smartwatch Fit	27600.00
5	2025-09-05	Arjun Mehta	Gaming Console	39600.00
6	2025-09-06	Priya Verma	Smartphone X100	23750.00
7	2025-09-07	Amit Sharma	Wireless Earbuds	10000.00

Solution:

```
CREATE TABLE customer_master
(
    customer_id VARCHAR(5) PRIMARY KEY,
    full_name VARCHAR(50) NOT NULL,
    city VARCHAR(30)
);

CREATE TABLE product_catalog
(
    product_id VARCHAR(5) PRIMARY KEY,
    product_name VARCHAR(50) NOT NULL,
    unit_price NUMERIC(10,2) NOT NULL
);

CREATE TABLE sales_orders
(
    order_id SERIAL PRIMARY KEY,
    product_id VARCHAR(5)
        REFERENCES product_catalog(product_id),

    quantity INT NOT NULL,

    customer_id VARCHAR(5)
        REFERENCES customer_master(customer_id),
```

```
    discount_percent NUMERIC(5,2),

    order_date DATE NOT NULL
);

INSERT INTO customer_master
(customer_id, full_name, city)
VALUES
('C1', 'Amit Sharma', 'Delhi'),
('C2', 'Priya Verma', 'Mumbai'),
('C3', 'Ravi Kumar', 'Bangalore'),
('C4', 'Neha Singh', 'Kolkata'),
('C5', 'Arjun Mehta', 'Hyderabad');

INSERT INTO product_catalog
(product_id, product_name, unit_price)
VALUES
('P1', 'Smartphone X100', 25000),
('P2', 'Laptop Pro 15', 65000),
('P3', 'Wireless Earbuds', 5000),
('P4', 'Smartwatch Fit', 30000),
('P5', 'Gaming Console', 45000);


INSERT INTO sales_orders
(product_id, quantity, customer_id, discount_percent,
order_date)
VALUES
('P1', 2, 'C1', 5, '2025-09-01'),
('P2', 1, 'C2', 10, '2025-09-02'),
('P3', 3, 'C3', 0, '2025-09-03'),
('P4', 1, 'C4', 8, '2025-09-04'),
('P5', 1, 'C5', 12, '2025-09-05'),
('P1', 1, 'C2', 5, '2025-09-06'),
('P3', 2, 'C1', 0, '2025-09-07');


CREATE OR REPLACE VIEW VW_DISCOUNT_AMOUNT AS
SELECT
  S.ORDER_ID,
  S.ORDER_DATE,
  C.FULL_NAME,
  P.PRODUCT_NAME,
  ROUND(
```

```
   (P.UNIT_PRICE * S.QUANTITY)
   -
   (P.UNIT_PRICE * S.QUANTITY) * S.DISCOUNT_PERCENT / 100
   , 2) AS FINAL_AMOUNT


FROM SALES_ORDERS S

JOIN CUSTOMER_MASTER C
ON S.CUSTOMER_ID = C.CUSTOMER_ID

JOIN PRODUCT_CATALOG P
ON S.PRODUCT_ID = P.PRODUCT_ID


SELECT * FROM VW_DISCOUNT_AMOUNT;


CREATE USER CLIENT_123
PASSWORD '12345';

GRANT SELECT ON VW_DISCOUNT_AMOUNT TO CLIENT_123;
```

(B)

An e-commerce company wants to regularly analyze the performance of its product categories.

The company stores product, category, customer, and order information in separate tables. To generate the category performance report, the database needs to perform multiple JOINs, GROUP BY, aggregate functions, DISTINCT counting, calculated values, and CASE statements.

Since the same report is generated frequently, executing the complete analytical query every time can increase database workload.

Create a PostgreSQL Materialized View named: mv_category_sales

Also demonstrate that when new order data is inserted, the Materialized View does not automatically reflect the change and must be refreshed.

Finally, use EXPLAIN ANALYZE to compare the performance of the original analytical query with the Materialized View query.

Output:

category_id	category_name	total_products_sold	total_orders	unique_customers	total_revenue	avg_revenue_per_order	category_performance
C1	Electronics	5	4	3	12000	3000.00	Excellent
C2	Clothing	4	3	2	10000	3333.33	Excellent
C3	Books	3	1	1	1500	1500.00	Low

Solution:

```
CREATE TABLE category_master (
    category_id VARCHAR(5) PRIMARY KEY,
    category_name VARCHAR(50) NOT NULL
);

CREATE TABLE product_master (
    product_id VARCHAR(5) PRIMARY KEY,
    product_name VARCHAR(50) NOT NULL,
    category_id VARCHAR(5)
        REFERENCES category_master(category_id)
);

CREATE TABLE customer_master (
    customer_id VARCHAR(5) PRIMARY KEY,
    customer_name VARCHAR(50) NOT NULL
);

CREATE TABLE order_details (
    order_id VARCHAR(5) NOT NULL,
    customer_id VARCHAR(5)
        REFERENCES customer_master(customer_id),
    product_id VARCHAR(5)
        REFERENCES product_master(product_id),
    quantity INT NOT NULL,
    unit_price NUMERIC(10,2) NOT NULL
);


INSERT INTO category_master
(category_id, category_name)
VALUES
('C1', 'Electronics'),
('C2', 'Clothing'),
```

```
('C3', 'Books');

INSERT INTO product_master
(product_id, product_name, category_id)
VALUES
('P101', 'Laptop', 'C1'),
('P102', 'Headphones', 'C1'),
('P201', 'Jacket', 'C2'),
('P202', 'Shoes', 'C2'),
('P301', 'SQL Book', 'C3');

INSERT INTO customer_master
(customer_id, customer_name)
VALUES
('U1', 'Aman'),
('U2', 'Priya'),
('U3', 'Rahul'),
('U4', 'Simran');

INSERT INTO order_details
(order_id, customer_id, product_id, quantity, unit_price)
VALUES
('O101', 'U1', 'P101', 1, 5000),
('O102', 'U2', 'P102', 2, 1000),
('O103', 'U1', 'P201', 1, 3000),
('O104', 'U3', 'P202', 2, 2000),
('O105', 'U4', 'P301', 3, 500),
('O106', 'U2', 'P101', 1, 5000),
('O107', 'U3', 'P201', 1, 3000),
('O108', 'U4', 'P102', 1, 1000);

EXPLAIN ANALYZE
SELECT
    CAT.CATEGORY_ID,
    CAT.CATEGORY_NAME,
    SUM(O.QUANTITY) AS TOTAL_PRODUCTS_SOLD,
    COUNT(DISTINCT O.ORDER_ID) AS TOTAL_ORDERS,
    COUNT(DISTINCT O.CUSTOMER_ID) AS UNIQUE_CUSTOMERS,
    SUM(O.QUANTITY * O.UNIT_PRICE) AS TOTAL_REVENUE,
    ROUND(
        SUM(O.QUANTITY * O.UNIT_PRICE)
        / COUNT(DISTINCT O.ORDER_ID),
        2
    ) AS AVERAGE_REVENUE_PER_ORDER,
```

```
    CASE
        WHEN SUM(O.QUANTITY * O.UNIT_PRICE) >= 10000
            THEN 'EXCELLENT'
        WHEN SUM(O.QUANTITY * O.UNIT_PRICE) >= 6000
            THEN 'GOOD'
        WHEN SUM(O.QUANTITY * O.UNIT_PRICE) >= 3000
            THEN 'AVERAGE'
        ELSE 'LOW'
    END AS CATEGORY_PERFORMANCE

FROM ORDER_DETAILS AS O

JOIN PRODUCT_MASTER AS P
    ON O.PRODUCT_ID = P.PRODUCT_ID

JOIN CATEGORY_MASTER AS CAT
    ON P.CATEGORY_ID = CAT.CATEGORY_ID

GROUP BY
    CAT.CATEGORY_ID,
    CAT.CATEGORY_NAME;

CREATE MATERIALIZED VIEW MV_CATEGORY_SALES AS
SELECT
    CAT.CATEGORY_ID,
    CAT.CATEGORY_NAME,
    SUM(O.QUANTITY) AS TOTAL_PRODUCTS_SOLD,
    COUNT(DISTINCT O.ORDER_ID) AS TOTAL_ORDERS,
    COUNT(DISTINCT O.CUSTOMER_ID) AS UNIQUE_CUSTOMERS,
    SUM(O.QUANTITY * O.UNIT_PRICE) AS TOTAL_REVENUE,
    ROUND(
        SUM(O.QUANTITY * O.UNIT_PRICE)
        / COUNT(DISTINCT O.ORDER_ID),
        2
    ) AS AVERAGE_REVENUE_PER_ORDER,
    CASE
        WHEN SUM(O.QUANTITY * O.UNIT_PRICE) >= 10000
            THEN 'EXCELLENT'
        WHEN SUM(O.QUANTITY * O.UNIT_PRICE) >= 6000
            THEN 'GOOD'
        WHEN SUM(O.QUANTITY * O.UNIT_PRICE) >= 3000
            THEN 'AVERAGE'
        ELSE 'LOW'
    END AS CATEGORY_PERFORMANCE
```

```
FROM ORDER_DETAILS AS O

JOIN PRODUCT_MASTER AS P
    ON O.PRODUCT_ID = P.PRODUCT_ID

JOIN CATEGORY_MASTER AS CAT
    ON P.CATEGORY_ID = CAT.CATEGORY_ID

GROUP BY
    CAT.CATEGORY_ID,
    CAT.CATEGORY_NAME;


EXPLAIN ANALYZE
SELECT * FROM MV_CATEGORY_SALES;

REFRESH MATERIALIZED VIEW MV_CATEGORY_SALES;
```